Simulación

Eje Temático Nº 2

Números Aleatorios

4to Nivel
Ingeniería en Sistemas de Información

Ing. Revilla, Sergio Andrés
Ing. Sofía Gaggi

Contenidos

Generación de números aleatorios 3
Introducción 3
¿Por qué no usar números «verdaderamente» aleatorios? 4
Un poco de historia: ¿de dónde salen estos generadores? 5
¿En la actualidad, qué se usa? 5
¿Qué hace bueno a un generador? Las propiedades deseables 6
Métodos de Generación 7
Generador por cuadrados medios 7
Limitaciones del método 8
Otras variantes históricas 8
Generadores Congruenciales Lineales (LCG) 8
Generador Congruencial Mixto 9
Generador Congruencial Multiplicativo 12
Pruebas estadísticas para números pseudoaleatorios 15
Mapa de las pruebas que veremos 16
1. Prueba de Promedios (presentación teórica) 17
Planteo formal 17
Criterio de decisión 18
¿Por qué solo la presentamos a modo teórico? 18
2. Prueba de Frecuencias (Chi-cuadrado) 19
Planteo formal 19
Procedimiento 19

Criterio de decisión 19

Criterios prácticos importantes 20

Ejemplo conceptual 21

3. Prueba de Kolmogorov-Smirnov 22

Planteo formal 22

Procedimiento 22

Criterio de decisión 22

Valores críticos de la tabla K-S (más usados) 23

Ejemplo conceptual 24

4. Prueba de Corridas arriba/debajo de la media 24

Planteo formal 25

Procedimiento 25

Criterio de decisión 25

Ejemplo conceptual ..... **¡Error! Marcador no definido.**

GENERACIÓN DE NÚMEROS ALEATORIOS

INTRODUCCIÓN

Toda simulación que represente un sistema real tiene una característica inevitable: **el mundo real no es determinístico**. Los clientes no llegan exactamente cada 3 minutos, las máquinas no fallan exactamente cada 100 horas, los pacientes no tardan exactamente lo mismo en ser atendidos. Hay **variabilidad**, y esa variabilidad es justamente lo que hace que valga la pena simular en lugar de calcular con una fórmula cerrada.

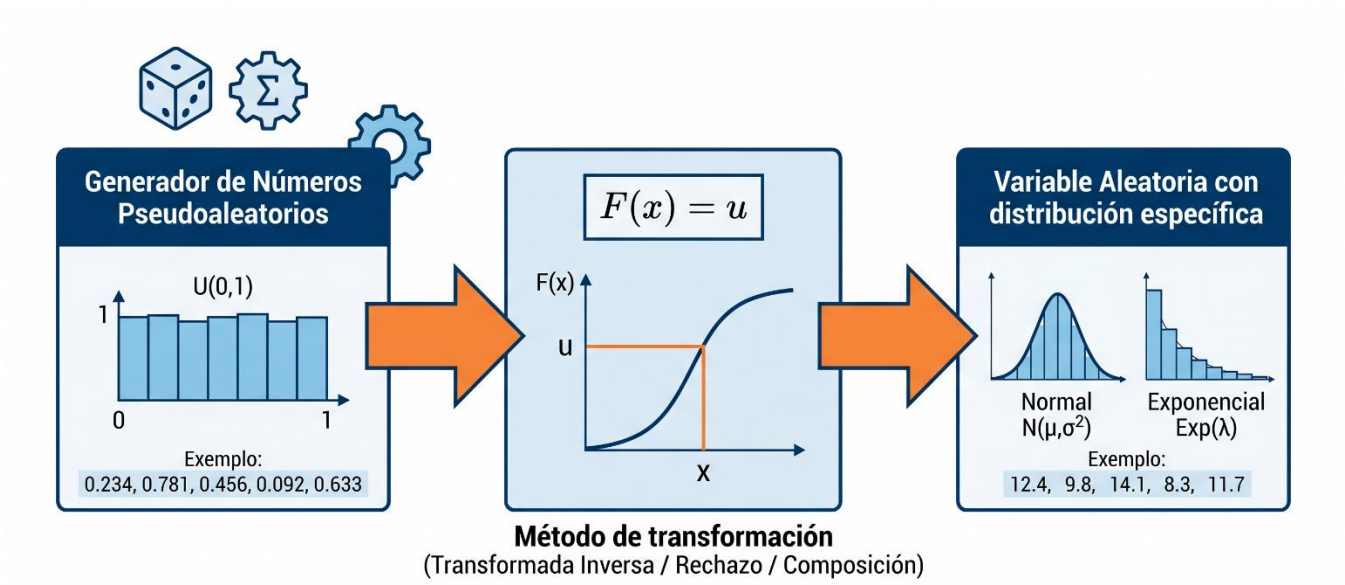
Para reproducir esa variabilidad dentro de una computadora necesitamos una fuente de aleatoriedad. Y acá aparece la primera tensión: una computadora es, por definición, una máquina **determinística**. Dada una entrada, siempre produce la misma salida. ¿Cómo le pedimos a algo determinístico que produzca algo aleatorio?

La respuesta histórica fue elegante: **los números pseudoaleatorios**. Son secuencias generadas por algoritmos determinísticos, pero que para fines prácticos se comportan como si fueran aleatorios. Cumplen dos propiedades fundamentales que necesitamos en simulación:

1. **Uniformidad** — están distribuidos uniformemente en el intervalo [0,1), es decir, todos los valores son igualmente probables.

2. **Independencia** — cada número generado no debe poder predecirse a partir de los anteriores.

Estas dos propiedades son la base sobre la que se apoya todo lo que vendrá después: la generación de variables aleatorias con cualquier distribución (Normal, Exponencial, Poisson), el modelado de tiempos entre llegadas, los tiempos de servicio, las decisiones probabilísticas dentro del modelo, las réplicas de simulación y el análisis estadístico de resultados.



¿POR QUÉ NO USAR NÚMEROS «VERDADERAMENTE» ALEATORIOS?

Existen fuentes de aleatoriedad física real (ruido térmico, decaimiento radioactivo, fluctuaciones cuánticas), pero **no son prácticas para simulación** por tres razones:

- **Son lentas.** Generar millones de números desde un fenómeno físico es órdenes de magnitud más lento que un algoritmo.
- **No son reproducibles.** Y la reproducibilidad es esencial: si quiero comparar dos configuraciones de mi modelo (por ejemplo, «3 cajas vs. 4 cajas en el supermercado»), necesito que ambas vean exactamente las mismas llegadas de clientes para que la comparación sea justa.
- **No se pueden depurar.** Si encuentro un comportamiento extraño en mi modelo, necesito poder repetir exactamente la misma corrida para encontrar el error.

Por eso, en simulación trabajamos casi exclusivamente con generadores pseudoaleatorios.

UN POCO DE HISTORIA: ¿DE DÓNDE SALEN ESTOS GENERADORES?

El problema de generar números aleatorios para cálculo científico no es nuevo. Tiene su origen en un lugar muy concreto: **el Proyecto Manhattan**, durante la Segunda Guerra Mundial. John von Neumann y Stanislaw Ulam, trabajando en Los Álamos en simulaciones de difusión de neutrones, necesitaban grandes cantidades de números aleatorios y se dieron cuenta de que las tablas impresas y las urnas con bolillas no escalaban.

Von Neumann propuso en 1946 el método de los **cuadrados medios** (middle-square method): tomar un número, elevarlo al cuadrado, extraer los dígitos centrales, y repetir. Es un método históricamente importante pero pésimo en la práctica: las secuencias colapsan rápidamente a ciclos cortos o al cero. Hoy se enseña como ejemplo de «lo que no hay que hacer», para que el alumno entienda por qué se necesitaron mejores métodos.

El salto cualitativo llegó en **1949**, cuando **Derrick Henry Lehmer**, matemático de la Universidad de California, presentó en un simposio en Harvard lo que hoy llamamos **generador multiplicativo congruencial** (Lehmer, 1951). La idea era usar la fórmula:

$$X_{(n+1)} = (a \cdot X_n) \bmod m$$

Esta fórmula tan simple resultó ser increíblemente poderosa y se convirtió en la **base de toda la familia de Generadores Congruenciales Lineales (LCG)** que vamos a estudiar. En 1958, Thomson y Rotenberg generalizaron la fórmula agregando una constante aditiva c, dando lugar al **generador congruencial mixto**:

$$X_{(n+1)} = (a \cdot X_n + c) \bmod m$$

Esta es la fórmula que vas a ver una y otra vez durante el resto de la unidad. Tiene sólo cuatro parámetros — a, c, m y la semilla X_o — pero la elección de esos parámetros decide si el generador es bueno o un desastre. De eso se ocupa el desarrollo que hace Coss Bu en detalle.

¿EN LA ACTUALIDAD, QUÉ SE USA?

Los Generadores Congruenciales Lineales (LCG), aunque siguen siendo educativamente valiosos y aún se usan en muchos contextos, **ya no son lo más utilizados**. Los simuladores y lenguajes modernos usan generadores más sofisticados:

- **Mersenne Twister (MT19937)** — desarrollado por Matsumoto y Nishimura en 1997, con un período enorme de 2^19937 − 1. Fue durante dos décadas el generador por defecto en Python, R, MATLAB, Excel y muchos otros.
- **PCG64 (Permuted Congruential Generator)** — desarrollado por O'Neill en 2014, es desde 2019 el generador por defecto en NumPy (a partir de la versión 1.17). Mejora estadísticamente al Mersenne Twister y usa menos memoria.
- **Generadores combinados (L'Ecuyer)** — combinan varios LCG simples para obtener períodos extremadamente largos y mejor independencia. Son los preferidos en bibliotecas profesionales de simulación.

Todos estos generadores modernos siguen siendo, en el fondo, **algoritmos determinísticos con una semilla inicial**. La idea de Lehmer de 1949 sigue viva en cada uno de ellos. Por eso entender el LCG no es estudiar una pieza de museo: es entender el ADN de toda la generación de aleatoriedad que usan los simuladores que ustedes van a usar en el Trabajo Práctico Integrador.

¿QUÉ HACE BUENO A UN GENERADOR? LAS PROPIEDADES DESEABLES

A continuación, describiremos las cinco propiedades que un buen generador debe cumplir:

1. **Uniformidad estadística** — los números generados deben distribuirse uniformemente en [0,1). No deben acumularse en zonas, ni tener «huecos».
2. **Independencia estadística** — no debe haber correlación entre números sucesivos. Si los grafico de a pares (X_n, X_(n+1)), no debe aparecer ningún patrón visible.
3. **Período largo** — la secuencia eventualmente se repite (porque hay finitos estados posibles en una computadora), pero el período debe ser lo suficientemente largo como para que en ninguna simulación práctica se llegue a repetir.
4. **Reproducibilidad** — dada la misma semilla, debe producir exactamente la misma secuencia. Esto es esencial para depurar y para comparar alternativas (Common Random Numbers).
5. **Eficiencia computacional** — debe ser rápido y consumir poca memoria. Una simulación industrial puede pedirle al generador miles de millones de números.

Estas propiedades no son negociables. Un generador que falla en cualquiera de ellas puede arruinar una simulación completa, llevando a conclusiones falsas sobre el sistema real.

Por eso, en la unidad siguiente vamos a estudiar **pruebas estadísticas** que permiten verificar empíricamente si un generador cumple estas propiedades. Y entender la fórmula del LCG nos va a permitir ver cómo la elección de a, c y m impacta directamente en cada una de estas propiedades.

MÉTODOS DE GENERACIÓN

GENERADOR POR CUADRADOS MEDIOS

El procedimiento es el siguiente:

1. Se toma una semilla X_o de N cifras (típicamente N=4).
2. Se eleva al cuadrado, obteniendo un número de hasta 2N cifras (si tiene menos, se completa con ceros a la izquierda).
3. Se extraen las N cifras centrales: ese es el siguiente número de la sucesión, X_{n+1}.
4. Se divide X_{n+1} entre 10^N para obtener el número uniforme U_n en [0,1).
5. Se repite el proceso usando X_{n+1} como nueva semilla.

Ejemplo: partiendo de $X_o = 7182$, generamos la siguiente secuencia:

n	X_n	X_n²	X_{n+1}	U_n
0	7182	51581124	5811	0.5811
1	5811	33767721	7677	0.7677
2	7677	58936329	9363	0.9363
3	9363	87665769	6657	0.6657
4	6657	44315649	3156	0.3156
5	3156	09960336	9603	0.9603

Nota sobre la tabla: cuando el cuadrado tiene menos de 2N cifras, se completa con ceros a la izquierda antes de extraer las centrales (ver fila n=5, donde $3156^2 = 9960336$ se trata como 09960336 para extraer 9603).

LIMITACIONES DEL MÉTODO

Aunque es elegante por su simplicidad, este método **presenta defectos graves** que lo hacen inviable en simulación profesional:

- Con ciertas semillas, la sucesión colapsa rápidamente al cero (por ejemplo: $0^2 = 0$, y a partir de ahí se queda en cero para siempre).
- Otras semillas generan ciclos cortos, repitiéndose después de unas pocas iteraciones.
- El período total es difícil de predecir a partir de la semilla.
- No supera las pruebas estadísticas de independencia.

Por estas razones, ya en la década del 50 el método fue desplazado por los generadores congruenciales lineales, que tienen un comportamiento mucho más predecible y controlable.

OTRAS VARIANTES HISTÓRICAS

Existen otras variantes del enfoque «extraer dígitos centrales», entre ellas el **método del Producto Medio** (que usa dos semillas que se multiplican) y el **método del Multiplicador Constante** (que multiplica la semilla por una constante fija). Estos métodos son refinamientos del Cuadrados Medios y comparten sus limitaciones de fondo: períodos impredecibles y dependencia fuerte de la elección inicial.

GENERADORES CONGRUENCIALES LINEALES (LCG)

Los Generadores Congruenciales Lineales fueron propuestos por **Derrick H. Lehmer** en 1949 y son, hasta hoy, la familia más estudiada y enseñada en simulación. Su forma general es:

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

Según los valores que tomen sus parámetros, hablamos de tres variantes:

- **Mixto**: cuando $c > 0$.
- **Multiplicativo**: cuando $c = 0$.
- **Aditivo**: variante histórica donde $a = 1$; hoy casi no se usa.

GENERADOR CONGRUENCIAL MIXTO

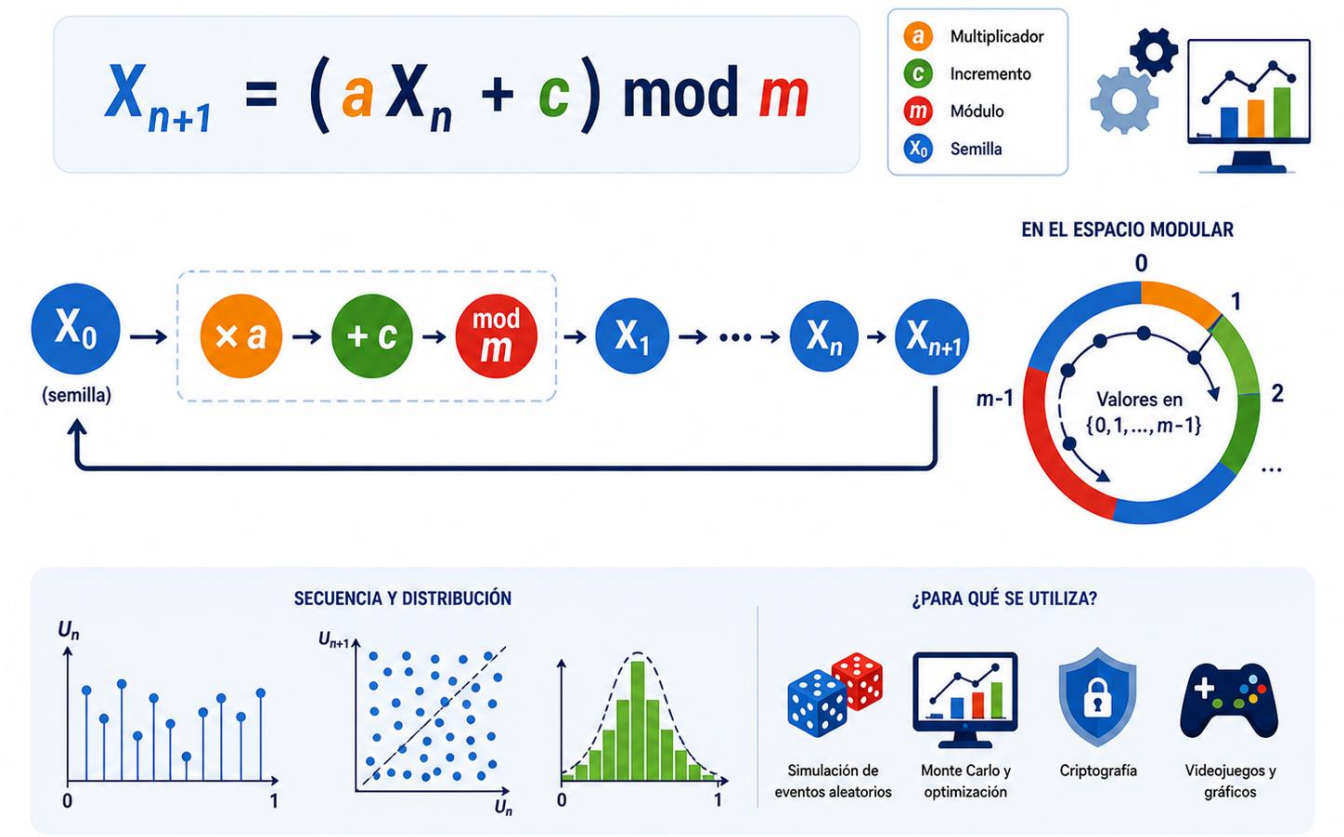
La relación de recurrencia es:

$$X_{(n+1)} = (a \cdot X_n + c) \bmod m$$

donde:

- **X_o:** semilla inicial ($X_o \geq 0$)
- **a:** multiplicador ($a > 0$)
- **c:** constante aditiva ($c > 0$)
- **m:** módulo ($m > a$, $m > c$, $m > X_o$)

El módulo **m** cumple un rol fundamental: define el universo de valores posibles que puede tomar la secuencia. Los valores generados son enteros en el conjunto {0, 1, 2, ..., m-1}, por lo tanto **m representa la cantidad máxima de valores diferentes que pueden producirse** antes de que la secuencia comience a repetirse.



¿Por qué dividir el resultado entre m?

La recurrencia produce **números enteros** en el rango {0, 1, ..., m-1}, pero en simulación necesitamos números **uniformes en el intervalo [0,1)**, porque esa es la entrada que requieren todos los métodos posteriores de generación de variables aleatorias (transformada inversa, rechazo, composición) y todos los simuladores comerciales.

Para llevar los enteros al intervalo [0,1) los escalamos dividiendo por m:

$$U_n = X_n / m$$

Así, los valores posibles del uniforme son {0, 1/m, 2/m, ..., (m-1)/m}, que para m grande aproxima perfectamente al continuo [0,1). Para m = 8 (como en el ejemplo siguiente) solo hay 8 valores posibles, lo que es muy pobre. Por ejemplo, para m = 2^{31} - 1 hay más de 2.000 millones de valores posibles, indistinguibles del continuo a fines prácticos.

RECOMENDACIONES PARA ELEGIR LOS PARÁMETROS

La calidad del generador depende críticamente de la elección de a, c y m. Sin entrar en demostraciones formales, los criterios prácticos son:

- **m debe ser grande**. Es el límite superior del período. En implementaciones reales suele elegirse como una potencia de 2 (m = 2^k) o un primo grande, según la arquitectura. En la práctica académica usamos m pequeño para poder seguir los cálculos a mano.
- **a, c y m deben elegirse para asegurar período completo**. Existen condiciones matemáticas conocidas (teorema de Hull-Dobell, 1962) que garantizan que el generador recorra los m valores posibles antes de repetirse. Las veremos en la ejercitación práctica.
- **X_0 no afecta el período si las condiciones anteriores se cumplen**. Solo determina el punto de la secuencia desde donde se empieza.

EJERCICIO RESUELTO

Generar la sucesión con los siguientes parámetros: **X_o = 4, a = 5, c = 7, m = 8.**

Aplicando la recurrencia **X_{n+1} = (5·X_n + 7) mod 8**:

n	X_n	5·X_n + 7	X_{n+1}	U_n = X_{n+1}/ m	U_n decimal
0	4	27	3	3/8	0.375
1	3	22	6	6/8	0.750
2	6	37	5	5/8	0.625
3	5	32	0	0/8	0.000
4	0	7	7	7/8	0.875
5	7	42	2	2/8	0.250
6	2	17	1	1/8	0.125
7	1	12	4	4/8	0.500
8	4	27	3	3/8	0.375

Observación importante: en la fila n=8 (resaltada) vemos que $X_8 = 3$, es decir, el mismo valor que X_1. A partir de acá la secuencia se repite. El período resultó ser **p = 8**, que coincide con m. Decimos entonces que este generador tiene **período completo**, lo cual es la característica deseable.

¿Período completo siempre? No. Podría pensarse que el período de todo generador congruencial es siempre igual a m, pero esto es **falso**. El período depende de los valores asignados a los parámetros. Existen condiciones específicas (que veremos a continuación) para que el generador alcance el período máximo.

CÓMO SE INTERPRETA CADA COLUMNA DEL EJERCICIO

- **n:** índice de la iteración.
- **X_n:** el entero actual; en n=0 es la semilla X_0.
- **5·X_n + 7:** el cálculo previo al módulo.
- **X_{n+1}:** el resto de dividir (5·X_n + 7) por 8. Este es el nuevo entero, que será X_n en la próxima fila.
- **U_n:** el número uniforme, expresado como fracción X_{n+1}/m.
- **U_n decimal:** el mismo número uniforme expresado como decimal. Este es el valor que efectivamente «usa» la simulación.

GENERADOR CONGRUENCIAL MULTIPLICATIVO

Es un caso particular del Mixto donde se elimina la constante aditiva (c = 0). Su recurrencia es:

$$X_{(n+1)} = (a \cdot X_n) \bmod m$$

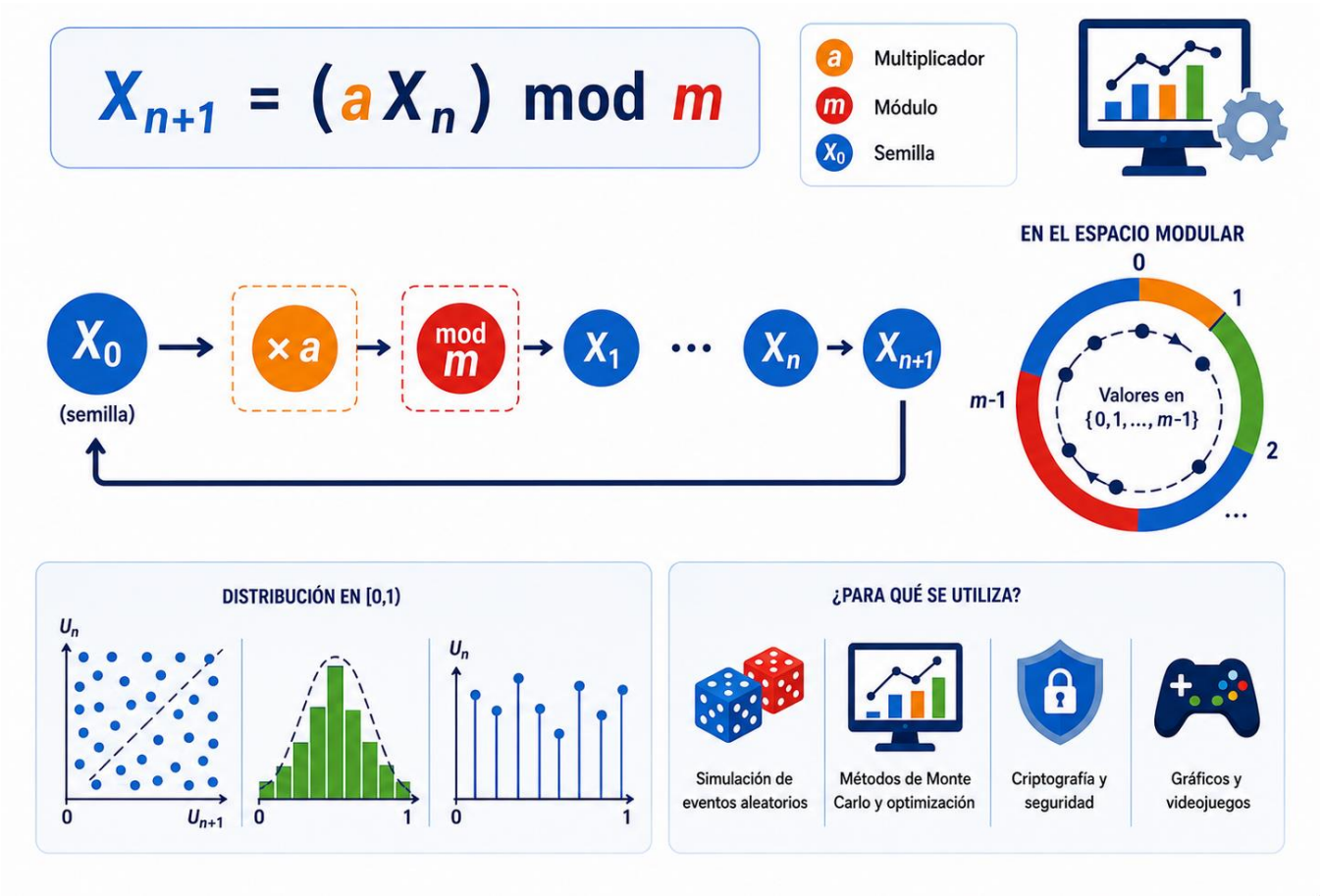
Históricamente es el más antiguo de la familia: fue exactamente el que propuso Lehmer en 1949. Su principal ventaja es la **eficiencia computacional**: al no tener la suma, requiere una operación menos por iteración, lo cual era crítico en las computadoras de los años 50 y sigue siendo relevante hoy cuando se generan miles de millones de números.

Su principal limitación es que **X_n = 0 colapsa la secuencia** (cualquier valor multiplicado por 0 sigue siendo 0). Por eso la semilla debe ser distinta de cero y el período máximo posible es m-1 (no m, como en el mixto). Cuando m es primo y a es una raíz primitiva módulo m, se alcanza efectivamente ese período máximo.

Ejemplo histórico: el generador **Park-Miller (1988)**, también llamado MINSTD, usa a = 16807 y m = 2^{31} - 1 = 2.147.483.647. Tiene un período de aproximadamente 2.000 millones de números y fue durante décadas el generador de referencia en lenguajes como C y Pascal.

RECOMENDACIONES PARA ELEGIR LOS PARÁMETROS (MULTIPLICATIVO)

- **Semilla $X_0 \neq 0$**, para evitar que la secuencia colapse.
- **m debe ser primo y grande**, por ejemplo un primo de Mersenne como $2^{31} - 1$.
- **a debe ser una raíz primitiva módulo m**. Esta es una condición algebraica que garantiza que la sucesión recorra los m-1 valores posibles antes de repetirse.
- **El período máximo es m-1**, no m, porque el valor 0 nunca puede aparecer en la sucesión.



ACLARACIÓN PRÁCTICA: CÓMO CALCULAR EL MÓDULO EN LA CALCULADORA

Una dificultad muy común en la ejercitación es que **muchas calculadoras científicas no incluyen una tecla directa para «mod»**. Repasamos acá tres formas de calcular el módulo, en orden de comodidad:

FORMA 1: CÁLCULO MENTAL PARA MÓDULOS PEQUEÑOS

Si los números son pequeños (que es lo habitual en los ejercicios de clase), el módulo se puede obtener mentalmente: **a mod b** es el **resto** de dividir a por b.

Ejemplo: 27 mod 8 = 3, porque 27 = 8·3 + 3.

FORMA 2: USANDO LA FÓRMULA A MOD B = A − B · ⌊A/B⌋

Funciona en cualquier calculadora, incluso las más básicas. El procedimiento es:

1. Calcular a/b en la calculadora. Si el resultado tiene decimales, **descartar la parte decimal** (no redondear: simplemente quedarse con la parte entera, lo que se llama «piso» o «truncamiento»).
2. Multiplicar ese entero por b.
3. Restarle ese producto al valor original a.

Ejemplo: 27 mod 8. Calculamos 27 / 8 = 3,375. Descartamos los decimales: 3. Multiplicamos: 3 · 8 = 24. Restamos: 27 - 24 = 3. Resultado: 27 mod 8 = 3. ✓

FORMA 3: USANDO LA TECLA ESPECÍFICA DE MOD O RESTO

Algunas calculadoras científicas y herramientas digitales tienen una función directa:

- **Casio fx-991 (Spanish/Plus/CW):** menú [OPTN] → opción «÷R» (división con resto). Se ingresa a ÷R b y el resultado muestra cociente y resto por separado.
- **Casio fx-82 (la más común en Argentina):** no tiene función directa. Hay que usar la Forma 2.
- **Excel / Google Sheets:** función =RESIDUO(27;8) en español o =MOD(27,8) en inglés.

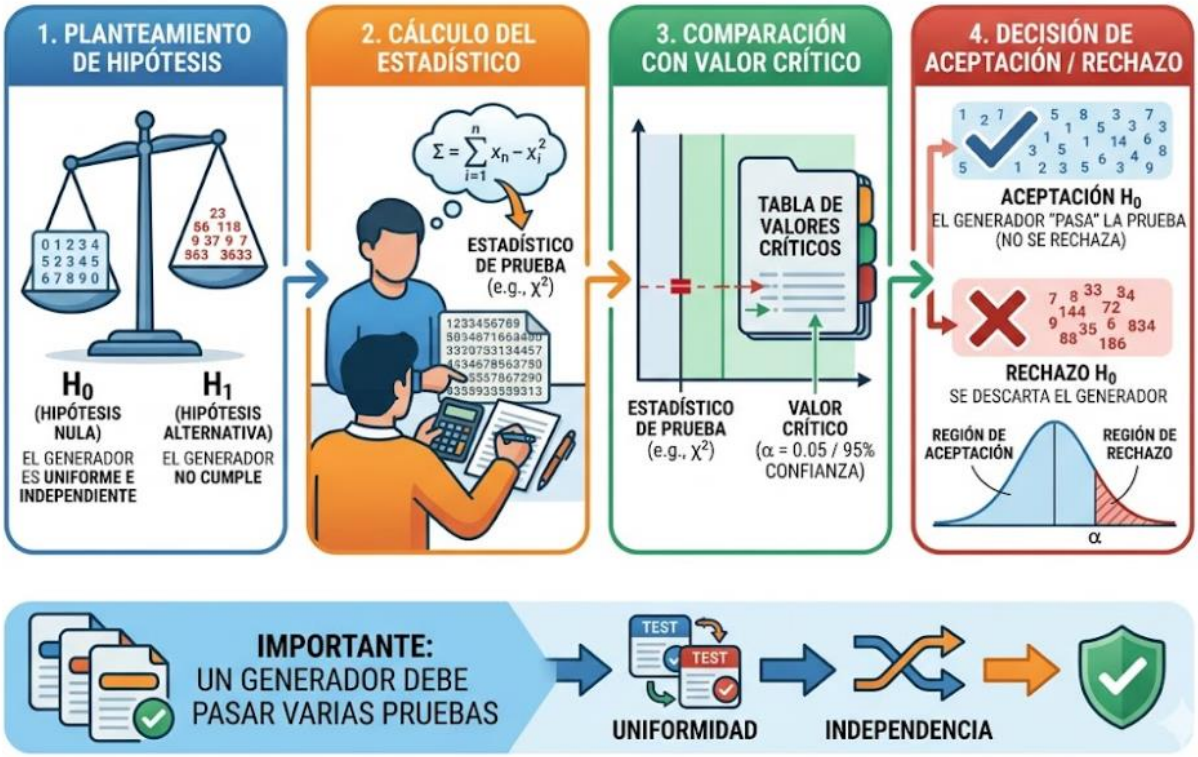
Recomendación para la práctica: en los ejercicios de clase la Forma 2 es la más segura porque funciona en cualquier calculadora. Una vez que el alumno se familiariza con el procedimiento, los cálculos son rápidos incluso para valores grandes.

PRUEBAS ESTADÍSTICAS PARA NÚMEROS PSEUDOALEATORIOS

En las unidades anteriores estudiamos los métodos para generar números pseudoaleatorios mediante algoritmos como los Métodos Congruenciales. Ahora bien, antes de usar un generador en una simulación real, necesitamos verificar que cumpla las dos propiedades fundamentales que vimos: uniformidad e independencia. Para ello aplicamos pruebas estadísticas, que no «demuestran» que el generador sea bueno, sino que permiten descartarlo si no cumple alguna propiedad esperada.

Antes de entrar a cada prueba, conviene fijar un marco común. Toda prueba estadística sigue el mismo esquema general:

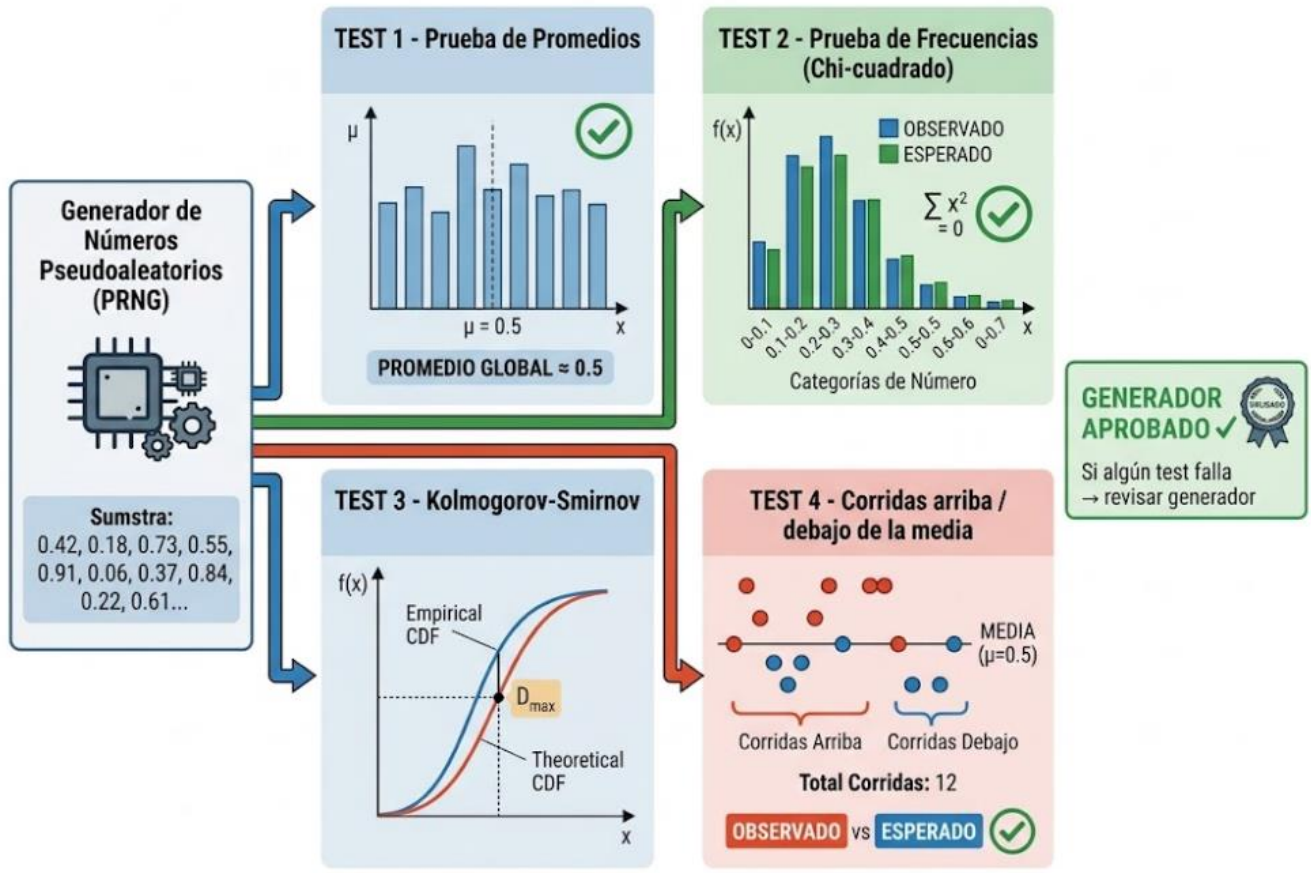
1. **Se plantean dos hipótesis:** una hipótesis nula H_0 (la afirmación a poner a prueba, típicamente «el generador es uniforme» o «los números son independientes») y una hipótesis alternativa H_1 (la negación de H_0).
2. **Se calcula un estadístico de prueba** a partir de la muestra de números generados.
3. **Se compara ese estadístico con un valor crítico** tabulado, que depende del nivel de significancia α elegido (típicamente α = 0.05, lo que equivale a un nivel de confianza del 95%).
4. **Se decide:** si el estadístico cae dentro de la región de aceptación, no se puede rechazar H_0 (el generador «pasa» la prueba). Si cae en la región de rechazo, se rechaza H_0 y se descarta el generador.



MAPA DE LAS PRUEBAS QUE VEREMOS

Prueba	¿Qué evalúa?	Tipo
Promedios	Que la media de la muestra esté cerca de 0.5	Teórica (introductoria)
Chi-cuadrado (χ^2)	Uniformidad de la distribución	Práctica
Kolmogorov-Smirnov	Uniformidad de la distribución acumulada	Práctica
Corridas arriba/debajo de la media	Independencia entre números sucesivos	Práctica

Las tres prácticas son las que aplicaremos efectivamente en los trabajos prácticos. La Prueba de Promedios la presentamos como introducción conceptual al esquema general de hipótesis estadísticas.



1. PRUEBA DE PROMEDIOS (PRESENTACIÓN TEÓRICA)

Esta es históricamente la primera prueba que se aplica a un generador. Su idea es muy intuitiva: si un generador produce números uniformes en [0,1), entonces el **promedio** de una muestra grande de esos números debería estar cerca de la media teórica de la distribución uniforme, que es 0.5.

PLANTEO FORMAL

Hipótesis nula H_0: los números generados provienen de una distribución uniforme con media $\mu = 0.5$.

Hipótesis alternativa H_1: la media de los números no es 0.5.

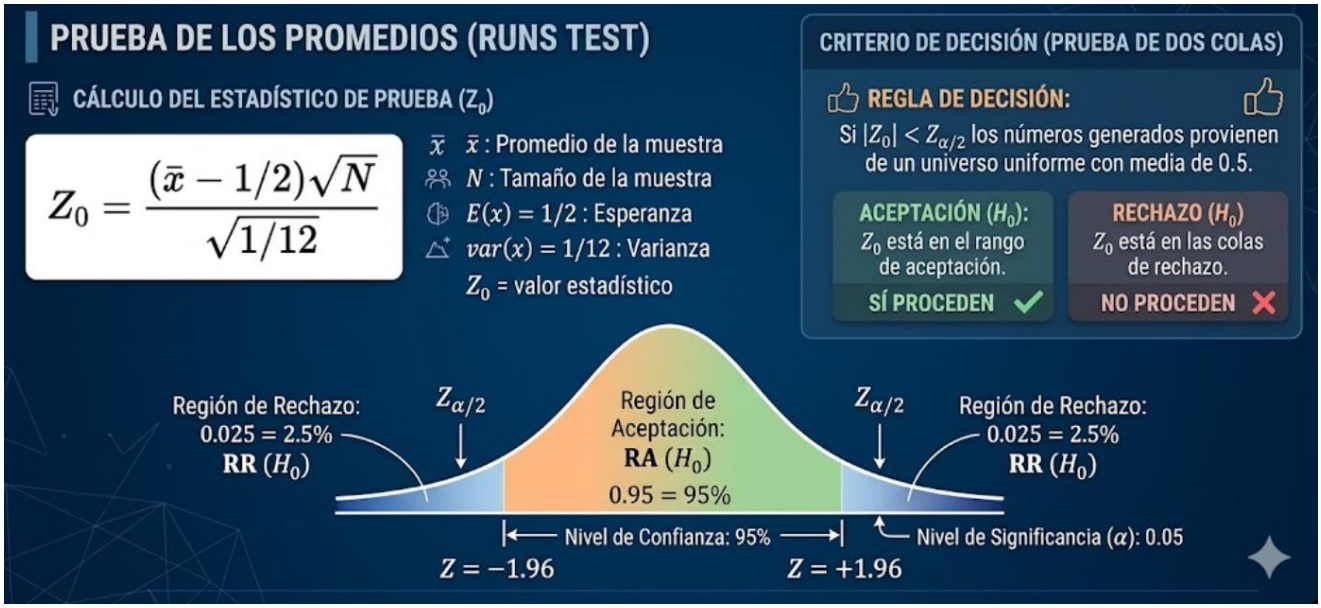
El estadístico de prueba se calcula como:

$$Z_0 = (\bar{X} - 0.5) \cdot \sqrt{N} / \sqrt{(1/12)}$$

donde:

- **$\bar{X}$:** promedio de la muestra
- **N:** tamaño de la muestra
- **0.5:** media esperada de U(0,1)
- **1/12:** varianza esperada de U(0,1)

El estadístico Z_0 se compara contra los valores críticos de la distribución normal estándar. Para un nivel de significancia $\alpha = 0.05$ (prueba de dos colas), los valores críticos son Z_{α/2} = ±1.96.



CRITERIO DE DECISIÓN

Si $|Z_o| < 1.96$ → no se puede rechazar H_o, los números **podrían provenir** de una distribución uniforme con media 0.5.

Si $|Z_o| \geq 1.96$ → se rechaza H_o: el generador presenta un sesgo en la media y debe descartarse.

¿POR QUÉ SOLO LA PRESENTAMOS A MODO TEÓRICO?

Aunque esta prueba es la más simple y se usa históricamente como primera verificación, tiene una limitación importante: **solo evalúa la media**, no la forma de la distribución.

Una secuencia perfectamente alternada como 0.1, 0.9, 0.1, 0.9, 0.1, 0.9… pasa esta prueba sin problemas (su media es 0.5), pero claramente no es uniforme ni aleatoria. Por eso, en la práctica profesional la Prueba de Promedios funciona como un primer filtro grueso (descarta generadores groseramente sesgados), pero nunca alcanza por sí sola para validar un generador.

En los trabajos prácticos no aplicaremos esta prueba, sino las tres siguientes, que son las que realmente permiten evaluar uniformidad e independencia.

2. PRUEBA DE FRECUENCIAS (CHI-CUADRADO)

Esta es la prueba clásica de uniformidad, propuesta por **Karl Pearson en 1900** y adaptada al contexto de simulación. Su idea es comparar las frecuencias **observadas** (cuántos de nuestros números cayeron en cada subintervalo) con las frecuencias **esperadas** (cuántos deberían caer si la distribución fuera realmente uniforme).

PLANTEO FORMAL

H_0: los números generados provienen de una distribución uniforme U(0,1).

H_1: los números no provienen de una distribución uniforme.

PROCEDIMIENTO

1. Se divide el intervalo [0,1) en **n subintervalos** de igual ancho (típicamente n = 8 o n = 10 para muestras pequeñas; ver criterio más abajo).
2. Se cuenta cuántos números cayeron en cada subintervalo: estas son las **frecuencias observadas (fo_i)**.
3. Se calcula la **frecuencia esperada (fe_i)** para cada subintervalo. Si la distribución es uniforme y todos los subintervalos tienen el mismo ancho, entonces fe_i = N / n para todos los i, donde N es el tamaño de la muestra.
4. Se calcula el estadístico χ^2_0:

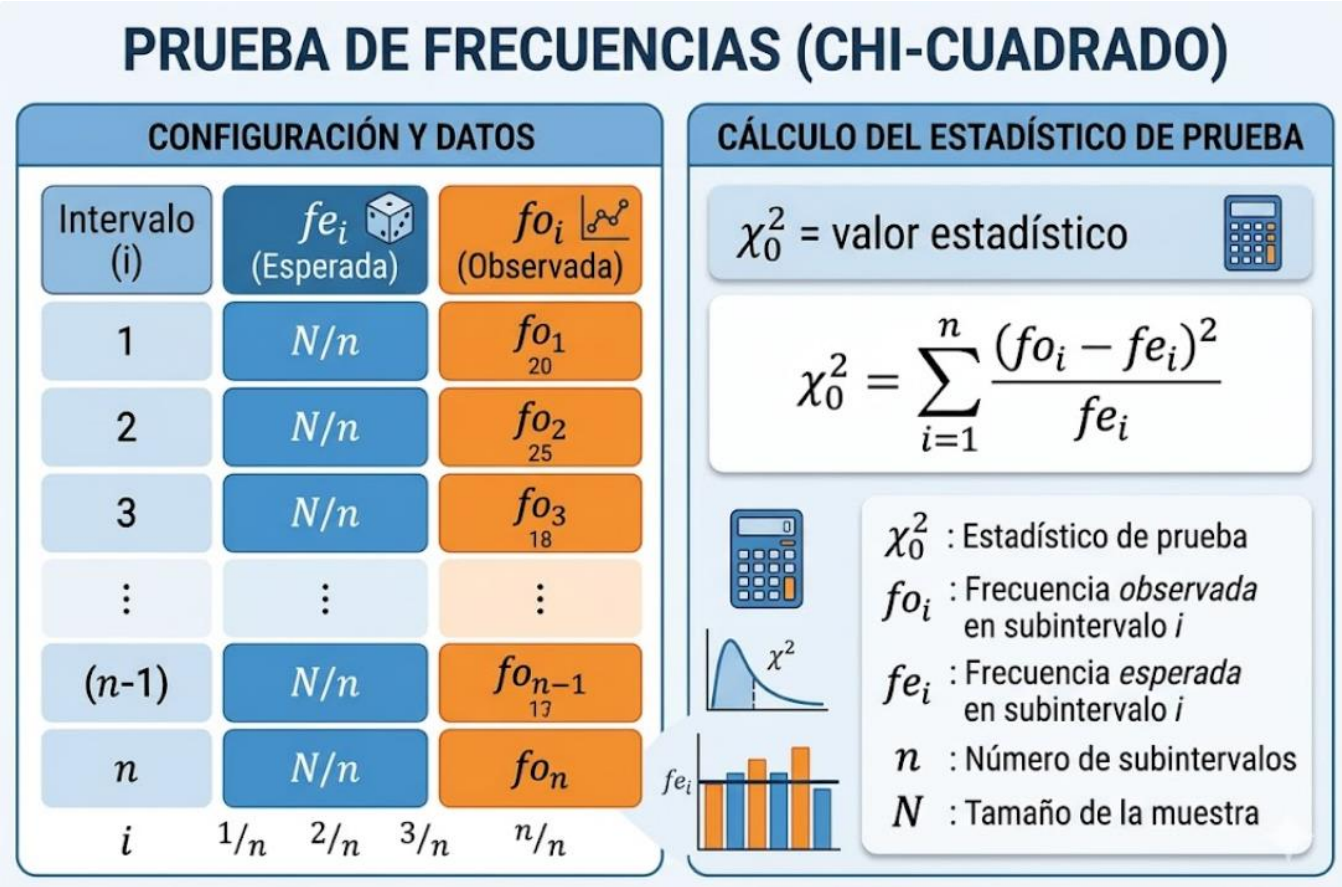
$$\chi^2_0 = \Sigma\ (fo_i - fe_i)^2\ /\ fe_i$$

5. Se compara χ^2_0 con el valor crítico $\chi^2_\{\alpha, n-1\}$ de la tabla chi-cuadrado con n−1 grados de libertad y nivel de significancia α.

CRITERIO DE DECISIÓN

Si $\chi^2_0 < \chi^2_\{\alpha, n-1\}$ → no se puede rechazar H_0, los números son consistentes con una distribución uniforme.

Si $\chi^2_0 \geq \chi^2_\{\alpha, n-1\}$ → se rechaza H_0, el generador no produce números uniformes.



CRITERIOS PRÁCTICOS IMPORTANTES

1) Elección del número de subintervalos. Una regla práctica común es n = √N, redondeada al entero más cercano. Para N = 30, esto sugiere n = 5 o 6 subintervalos. Algunos autores recomiendan n ≥ 5 y a la vez asegurar que la frecuencia esperada por intervalo sea suficiente (ver punto siguiente).

2) Regla de las frecuencias esperadas mínimas. La prueba χ^2 requiere que **fe_i ≥ 5** en cada subintervalo. Si algún subintervalo no cumple esta condición, hay que reducir n (agrupar subintervalos) hasta que todos cumplan la regla. Esto es especialmente importante con muestras pequeñas.

3) Grados de libertad. Son siempre n − 1 (donde n es el número de subintervalos, no el tamaño de la muestra). Para n = 5, los grados de libertad son 4.

EJEMPLO CONCEPTUAL

Supongamos que generamos N = 30 números pseudoaleatorios y los dividimos en n = 5 subintervalos de ancho 0.2. Aplicando la regla, fe_i = 30 / 5 = 6 para cada subintervalo (cumple fe_i ≥ 5).

Tabla de frecuencias hipotéticas:

Subintervalo i	Rango	fe_i	fo_i	$(fo_i - fe_i)^2 / fe_i$
1	[0.0, 0.2)	6	7	0.167
2	[0.2, 0.4)	6	5	0.167
3	[0.4, 0.6)	6	8	0.667
4	[0.6, 0.8)	6	4	0.667
5	[0.8, 1.0)	6	6	0.000
Total	—	30	30	$\chi^2_o = 1.668$

Análisis: para α = 0.05 y grados de libertad = 5 − 1 = 4, el valor crítico de tabla es $\chi^2_\{0.05, 4\} = 9.49$. Como $\chi^2_o = 1.668 < 9.49$, **no se puede rechazar H_o**: los números son consistentes con una distribución uniforme.

3. PRUEBA DE KOLMOGOROV-SMIRNOV

Esta prueba fue desarrollada por los matemáticos rusos **Andrei Kolmogorov (1933) y Nikolai Smirnov (1948)**, y resuelve algunas limitaciones de la prueba χ^2:

- No requiere agrupar los datos en subintervalos (cada número se trata individualmente).
- Es más potente que χ^2 para muestras pequeñas ($n < 50$).
- No depende de decisiones arbitrarias sobre cuántos intervalos usar.

La idea es comparar directamente la función de distribución acumulada empírica de la muestra contra la función de distribución acumulada teórica de la uniforme U(0,1).

PLANTEO FORMAL

H_0: los números generados provienen de una distribución uniforme U(0,1).

H_1: los números no provienen de una distribución uniforme.

PROCEDIMIENTO

1. Generar N números pseudoaleatorios uniformes: $x_1, x_2, ..., x_N$.
2. Ordenarlos en forma **ascendente**: $x_(1) \leq x_(2) \leq ... \leq x_(N)$.
3. Calcular la distribución acumulada empírica para cada número ordenado:

$$F_N(x_(i)) = i / N$$

donde i es la posición que ocupa el número x_(i) en el vector ordenado.

4. Calcular el estadístico de Kolmogorov-Smirnov:

$$D_N = \text{máx} \mid F_N(x_(i)) - x_(i) \mid \quad \text{para toda i}$$

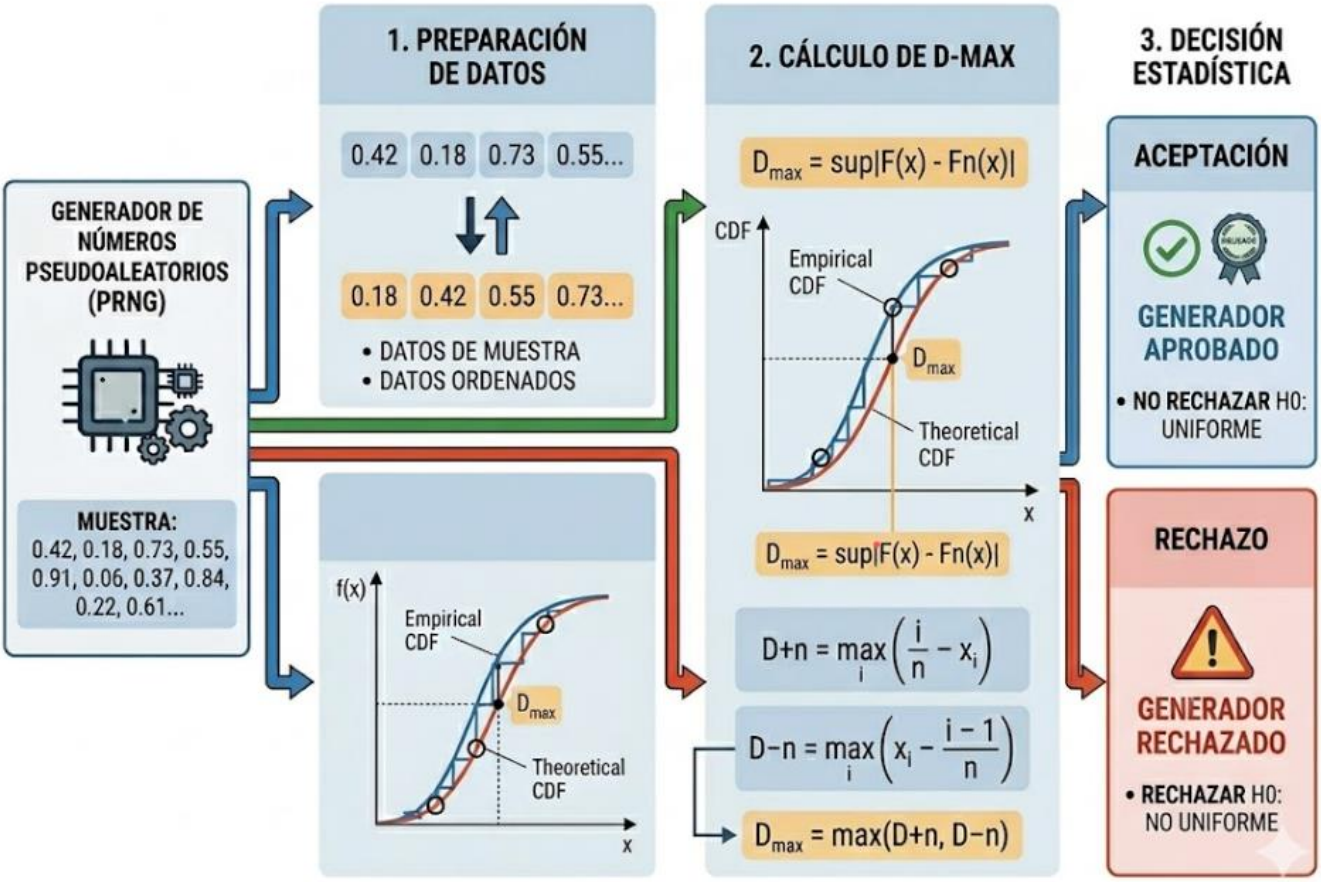
Es decir, se calcula la diferencia absoluta entre la acumulada empírica F_N(x_(i)) y el valor del número x_(i) (que es la acumulada teórica de U(0,1) en ese punto), y se queda con la diferencia máxima.

5. Comparar D_N con el valor crítico d_{α, N} de la tabla de Kolmogorov-Smirnov.

CRITERIO DE DECISIÓN

Si $D_N < d_\{\alpha, N\}$ → no se puede rechazar H_0.

Si $D_N \geq d_\{\alpha, N\}$ → se rechaza H_0.



VALORES CRÍTICOS DE LA TABLA K-S (MÁS USADOS)

N (tamaño)	α = 0.10	α = 0.05	α = 0.01
10	0.369	0.409	0.486
15	0.304	0.338	0.404
20	0.265	0.294	0.352
25	0.238	0.264	0.317
30	0.218	0.242	0.290
35	0.202	0.224	0.269
40	0.189	0.210	0.252
> 35	1.22/√N	1.36/√N	1.63/√N

Para nuestro caso (N = 30, α = 0.05): d_{0.05, 30} = 0.242.

EJEMPLO CONCEPTUAL

Supongamos que generamos N = 30 números, los ordenamos y construimos la tabla siguiente (mostramos solo algunas filas representativas):

i	x_(i)	F_N = i/N	|F_N − x_(i)|
1	0.031	0.033	0.002
2	0.058	0.067	0.009
...	...	...	...
15	0.487	0.500	0.013
...	...	...	...
22	0.681	0.733	0.052
...	...	...	...
30	0.972	1.000	0.028

Análisis: Supongamos que el valor máximo de la columna |F_N − x_(i)| resultó D_N = 0.052, observado en la fila i=22. Como D_N = 0.052 < 0.242 = d_{0.05, 30}, **no se puede rechazar H_0**: los números son consistentes con una distribución uniforme.

> **Detalle técnico:** Algunos autores calculan **dos diferencias por fila** (la diferencia hacia arriba |i/N − x_(i)| y la diferencia hacia abajo |(i−1)/N − x_(i)|) y toman el máximo entre ambas, lo que se conoce como **estadístico K-S bilateral**. En este apunte usamos la versión simplificada que es suficiente para detectar desvíos significativos de la uniformidad y es la que figura en Coss Bu.

4. PRUEBA DE CORRIDAS ARRIBA/DEBAJO DE LA MEDIA

Las tres pruebas anteriores evalúan la **uniformidad** de los números, pero **ninguna evalúa la independencia**. Una secuencia uniforme pero predecible (por ejemplo, 0.1, 0.2, 0.3, ..., 0.9, 0.1, 0.2, ...) pasaría las pruebas anteriores sin problemas, aunque claramente no sea aleatoria.

Para detectar dependencias entre números sucesivos usamos pruebas de **corridas**, también llamadas pruebas de rachas. La idea es contar las **rachas** (secuencias consecutivas de valores con una propiedad común) y comparar el número observado con el esperado bajo aleatoriedad.

Existen varias variantes; la que usaremos es la más simple: contar rachas de números arriba y debajo de la media (0.5).

PLANTEO FORMAL

H_0: los números generados son independientes entre sí.

H_1: los números presentan dependencia (no son independientes).

PROCEDIMIENTO

1. Para cada número de la secuencia, asignar el símbolo «+» si está por encima de 0.5 y «−» si está por debajo (los iguales a 0.5 se descartan o se asignan al grupo de la mayoría).
2. Identificar las **corridas**: cada secuencia consecutiva de símbolos iguales es una corrida. Por ejemplo, la secuencia ++−−−+−++− tiene 6 corridas (++ / −−− / + / − / ++ / −).
3. Contar:

- **n_1:** cantidad de números arriba de 0.5
- **n_2:** cantidad de números debajo de 0.5
- **C:** cantidad total de corridas observadas

4. Calcular el número esperado de corridas μ_C y su desviación estándar σ_C bajo la hipótesis de independencia:

$$\mu_C = (2 \cdot n_1 \cdot n_2) / (n_1 + n_2) + 1/2$$

$$\sigma^2_C = [2 \cdot n_1 \cdot n_2 \cdot (2 \cdot n_1 \cdot n_2 - n_1 - n_2)] / [(n_1 + n_2)^2 \cdot (n_1 + n_2 - 1)]$$

5. Calcular el estadístico Z_0:

$$Z_0 = (C - \mu_C) / \sigma_C$$

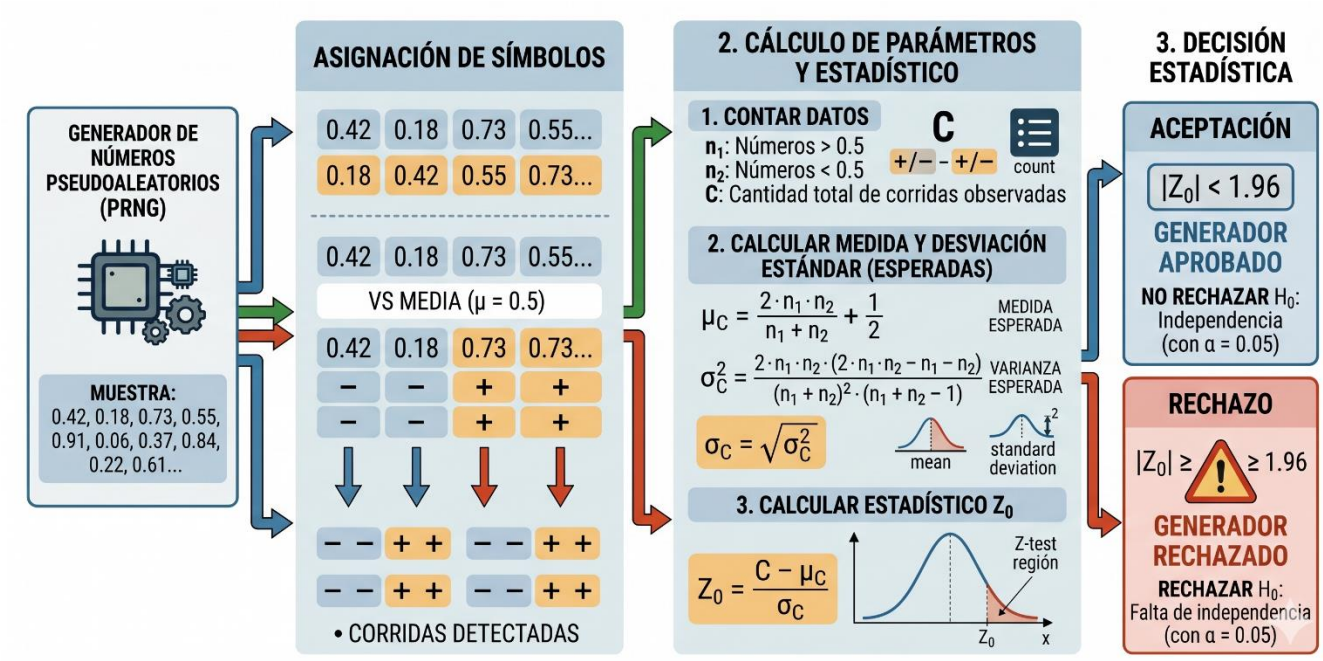
6. Comparar $|Z_0|$ con el valor crítico de la distribución normal estándar. Para α = 0.05, el valor crítico es Z_{α/2} = 1.96.

CRITERIO DE DECISIÓN

Si $|Z_0| < 1.96$ → no se puede rechazar H_0, los números son consistentes con una secuencia independiente.

Si $|Z_0| \geq 1.96$ → se rechaza H_0, hay evidencia de dependencia entre números sucesivos.

Interpretación intuitiva: si hay **muy pocas corridas**, significa que los números tienden a agruparse (varios consecutivos arriba, varios consecutivos debajo): hay tendencia o correlación positiva. Si hay **demasiadas corridas**, significa que los números alternan en exceso: hay anti-correlación. Ambos casos son indicios de no-independencia.



EJEMPLO CONCEPTUAL

Supongamos que un generador entregó los siguientes N = 20 números. A cada uno le asignamos «+» si supera 0.5 y «−» si está por debajo:

x_i	0.62	0.88	0.31	0.74	0.91	0.55	0.20	0.43	0.67	0.12
signo	+	+	−	+	+	+	−	−	+	−
x_i	0.39	0.81	0.58	0.49	0.26	0.70	0.84	0.33	0.15	0.77
signo	−	+	+	−	−	+	+	−	−	+

Separando la secuencia en bloques de signos iguales, identificamos las corridas (cada bloque es una corrida):

$$+ + \mid - \mid + + + \mid - - \mid + \mid - - \mid + + \mid - - \mid + + \mid - - \mid +$$

La secuencia presenta 11 bloques, de modo que C = 11. Contamos además:

- **n_1 =** 11 (números arriba de 0.5)
- **n_2 =** 9 (números debajo de 0.5)
- **C =** 11 corridas observadas (los 11 bloques identificados)

Calculamos el valor esperado y la desviación bajo independencia:

$$\mu_C = (2 \cdot 11 \cdot 9) / (11 + 9) + 1/2 = 198 / 20 + 0.5 = 10.4$$

$$\sigma^2_C = [2 \cdot 11 \cdot 9 \cdot (2 \cdot 11 \cdot 9 - 20)] / [20^2 \cdot 19] = [198 \cdot 178] / 7600 \approx 4.637$$

$$\sigma_C = \sqrt{(4.637)} \approx 2.153$$

$$Z_0 = (11 - 10.4) / 2.153 \approx 0.279$$

Análisis: como $|Z_0|$ = 0.279 < 1.96, **no se puede rechazar H_0**: la cantidad de corridas observada es muy cercana a la esperada, de modo que los números son consistentes con una secuencia independiente.